

Tool Results, Errors and Enforcement

Two kinds of failure, why the difference matters, and the one question that decides where a rule belongs.

A tool error is a message to **the model**. Not to you, and not to your logs.

Two kinds of failure

PROTOCOL ERROR

The request was broken

- A JSON-RPC error object with a code
- Unknown tool, malformed request
- The tool never ran at all
- The model usually never sees it

TOOL EXECUTION ERROR

The tool ran and failed

- A normal result with isError set to true
- Validation, API failure, business logic
- The tool existed and was invoked
- The model reads it and can self-correct

THE TIEBREAKER

The whole point of the split is what happens next. One the model can fix. The other it cannot.

Which one is which

WHAT HAPPENED	HOW IT SHOULD BE REPORTED
The tool name does not exist	Protocol error
The request was malformed at the protocol level	Protocol error
A parameter failed validation	isError: true
The upstream API returned a 500	isError: true
A business rule rejected the operation	isError: true
The database was locked	isError: true
The search matched nothing	Neither. That is a successful empty result.

ON THE EXAM

Almost everything is a tool execution error. Protocol errors are for requests that never reached a working handler.

Why validation errors belong in the result

- ◆ **As a protocol error, the model never sees the message.** It gets a generic failure and no way to correct itself.
- ◆ **As isError, it reads the message** and can retry with the value fixed.
- ◆ **The MCP specification clarified this explicitly.** Input validation is a tool execution error, not a protocol error.
- ◆ **Getting this wrong produces an agent that fails repeatedly on the same call,** with no idea why.

ON THE EXAM

This is the single most common MCP implementation mistake, and it turns a self-correcting agent into a stuck one.

What a good error message contains

- ◆ **What went wrong**, in plain language rather than a code.
- ◆ **Which field or input** caused it. Naming it is half the fix.
- ◆ **The expected format, with an example.** "start_date must be YYYY-MM-DD, e.g. 2026-09-01".
- ◆ **What to do next**, including which tool to call if this one cannot help.
- ◆ **Not a stack trace.** Internal paths and database details help nobody and leak plenty.

ON THE EXAM

Write the error as an instruction. The reader is an agent deciding its next move, not an engineer reading a log.

The same failure, two messages

MESSAGE	WHAT THE AGENT DOES NEXT
"Error: invalid input"	Retries the same call. Fails again. Eventually gives up or invents something.
"start_date must be YYYY-MM-DD. Received '1st Sept'. Example: 2026-09-01."	Reformats the date and calls again successfully.

ON THE EXAM

Both are accurate. Only one is useful. Accuracy is not the standard here.

Retryable or not

- ◆ **Transient: rate limits, timeouts, a 503.** Retry with backoff. It may work in ten seconds.
- ◆ **Deterministic: bad input, missing permission, a rule violation.** Retrying identically fails identically.
- ◆ **Say which it is in the error itself**, with a category or a retryable flag alongside the message.
- ◆ **The agent cannot tell them apart from prose.** Do not make it guess from the wording.

ON THE EXAM

"Every attempt fails the same way" in a stem means deterministic. No retry policy fixes a deterministic failure.

Do not swallow the error

- ◆ **Catching isError in your client code and handling it there** hides it from the thing that could fix it.
- ◆ **Pass the error content back to the model** as the tool result and let it reason.
- ◆ **Retry by sending the error back**, not by resending the same request blind.
- ◆ **Cap the attempts.** Two or three, then escalate. An uncapped correction loop is a cost incident.

ON THE EXAM

Blind retry and informed retry look identical in a diagram. Only one of them converges.

Empty is not an error

- ◆ **A search that matched nothing succeeded.** It answered the question, and the answer was none.
- ◆ **Reporting it as an error** makes the agent retry, rephrase, and eventually conclude the tool is broken.
- ◆ **Return an empty result with context.** What was searched, and what might widen it.
- ◆ **The same applies to a legitimate zero.** No outstanding invoices is a fact, not a failure.

ON THE EXAM

"No results found for X in Y. Try broadening the date range." is a successful result doing useful work.

What not to put in an error

- ◆ **Stack traces, internal paths, database details.** They reach the model's context and often the user.
- ◆ **Whether a record exists but is forbidden.** That distinction enables enumeration.
- ◆ **Raw upstream errors, unsanitised.** Whatever the vendor put in there is now in your conversation.
- ◆ **Enough to act on, and nothing more.** Sanitising is part of designing the output.

ON THE EXAM

Error text is tool output. Everything you would sanitise in a response applies here too.

Now the question that decides questions in four domains: **where does a rule live?**

Four places a rule can live

WHERE	WHAT IT GIVES YOU
The system prompt	Persuasion. Works most of the time. Fails when the agent is confused.
CLAUDE.md or a memory file	The same thing. It is context, loaded into the same window.
Inside the tool	Enforcement. The check runs on every call, whatever the agent intended.
A hook or a permission rule	Enforcement, outside the tool. Fires on the event and can block it.

ON THE EXAM

The top two are context. The bottom two are code. Only code can guarantee anything.

Describing a limit, and enforcing one

IN THE DESCRIPTION

"Refunds above £500 require approval"

- Helps the agent make good decisions
- Genuinely reduces the failure rate
- Still a sentence the agent may reason past
- Cannot be shown to an auditor as a control

IN THE TOOL

The tool rejects amounts over £500

- The check runs on every single call
- Independent of prompt, phrasing or jailbreak
- Returns an error the agent can act on
- Testable, and provable

THE TIEBREAKER

Do both. The description prevents the attempt; the check prevents the outcome. Only the second is a guarantee.

Tool-internal enforcement in practice

- ◆ **Validate before acting**, and return a clear `isError` rather than throwing.
- ◆ **Check authorisation inside the tool**, not in the prompt that describes it.
- ◆ **Make side effects idempotent** where you can, so a repeat is harmless.
- ◆ **Cap what the tool can do**, so a limit is a property of the capability rather than of the request.
- ◆ **Never accept a flag from the agent as authorisation**. A parameter the model sets is not a control.

ON THE EXAM

The last point catches people. "confirmed: true" is set by the same agent you are trying to constrain.

When the rule cannot live in the tool

- ◆ **You do not own the tool.** A third-party MCP server is not yours to modify.
- ◆ **The rule spans several tools.** Never write to production, whichever tool is doing the writing.
- ◆ **Then use a hook or a permission rule,** which fires on the event from outside.
- ◆ **Permissions remove the capability; hooks inspect and decide.** Module 9 covers both properly.

ON THE EXAM

Same principle, different layer. If the tool is not yours, move the control up rather than back into the prompt.

Reading the stem: errors and enforcement

WHAT THE STEM SAYS	WHAT IT WANTS
"retrying produces identical failures"	Deterministic. Fix the input or the structure, not the retry policy.
"the agent gives up after a failed call"	The error told it nothing actionable.
"repeatedly calls the same tool with the same bad value"	Validation reported as a protocol error, or an unhelpful message.
"decides the tool is broken when nothing matches"	Empty results reported as errors.
"must never exceed X, regardless of prompting"	Inside the tool, or a hook. Never the description alone.
"we cannot modify the third-party server"	A hook or a permission rule.

ON THE EXAM

The enforcement rows are the highest-value pattern in the exam. The qualifier does all the work.

Module 6 in five lines

- ◆ Protocol errors are for broken requests. Everything else is `isError`, so the model can read and fix it.
- ◆ Write errors as instructions: which field, what format, an example, what to do next.
- ◆ Say whether a failure is retryable. The agent cannot infer it from prose.
- ◆ An empty result is a success. Reporting it as an error breaks the agent's trust in the tool.
- ◆ Descriptions persuade. Tools, hooks and permissions enforce. "Guarantee" always means the second group.

ON THE EXAM

Seven scenarios next. Three are error design, three are enforcement, and one is neither.

An MCP server exposes a booking tool. When a date parameter arrives in the wrong format, the server raises an exception that the SDK converts into a JSON-RPC error.

Production traces show the agent calling the tool with the same malformed date up to five times, then abandoning the task and telling the user booking is unavailable.

The date the user supplied was "next Tuesday".

What is the root cause?

- ☐ A The agent should parse relative dates before calling the tool
- ☐ B The validation failure is reported as a protocol error, so the agent never sees what was wrong
- ☐ C The retry limit is too low for the agent to converge on a valid date
- ☐ D The tool should accept natural language dates

◆ PAUSE AND DECIDE

What is the root cause?

- A

 Agent should parse dates
It would if it knew the expected format. Nothing has told it what the format is.
- B

 Reported as a protocol error
Protocol errors are invisible to the model. It retries identically because it has learned nothing.
- C

 Retry limit too low
Five identical attempts. A sixth would also be identical. More retries of a blind retry is not convergence.
- D

 Accept natural language
Convenient, and it moves the parsing burden into the tool. The next malformed input still fails invisibly.

THE SIGNAL

"The same malformed date up to five times" is the tell. Identical repeats mean the agent received no information.

A refund tool is documented as: "Issues a customer refund. Refunds above £500 require manager approval and must not be issued directly."

Over six months, eleven refunds above £500 were issued directly. Review shows the agent had been given plausible reasons by customers in each case, and the system prompt also states the limit.

The compliance team needs to demonstrate that this cannot happen.

What satisfies the requirement?

- ☐ A Add the limit to CLAUDE.md as well, so it appears in three places
- ☐ B Reject amounts over £500 inside the tool, returning an error that directs the agent to the approval tool
- ☐ C Add a `requires_approval` parameter that the agent must set to true for amounts over £500
- ☐ D Add few-shot examples to the system prompt showing correct refusals above the limit

◆ PAUSE AND DECIDE

What satisfies the requirement?

- A

 Also in CLAUDE.md
A third copy of the same sentence in the same context window. Three persuasions are not an enforcement.
- B

 Reject inside the tool
The check runs on every call. Plausible reasons cannot move it, and compliance can test it.
- C

 requires_approval parameter
Set by the agent being persuaded. A flag the model controls is not a control.
- D

 Few-shot refusals
Improves the rate. "Demonstrate that this cannot happen" is not a rate.

THE SIGNAL

"Given plausible reasons" is the planted detail. It tells you the failure mode is persuasion, so no amount of prose helps.

A research agent uses a document search tool. When a query matches no documents, the tool returns an error: "Search failed: no matching documents."

Engineers notice the agent runs three or four progressively broader searches after an empty result, then stops using the search tool for the rest of the session and answers from memory instead.

What is the most effective change?

- ☐ A Return a successful empty result stating what was searched and suggesting how to broaden it
- ☐ B Instruct the agent in the system prompt that empty searches are normal and not failures
- ☐ C Have the tool automatically broaden the query when nothing matches
- ☐ D Suppress the error and return an empty array with no message

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Successful empty result with context
Fixes the miscategorisation and gives the agent something to do next.
- B

 Explain it in the prompt
Asks the agent to disbelieve the tool. The tool says failed; the prompt says ignore that.
- C

 Auto-broaden the query
The agent now cannot tell an exact match from a widened one. It hides the boundary it needs to see.
- D

 Empty array, no message
Better than an error, and it gives no context. Half the fix.

THE SIGNAL

"Stops using the tool" is the giveaway. Repeated failures taught the agent the tool is unreliable, so it routed around it.

An agent integrates a third-party MCP server for a payments provider. The team has a hard rule: no tool may be called against the production environment outside a change window.

The MCP server is maintained by the vendor and cannot be modified. It exposes eight tools, four of which can affect production.

The current approach is a system prompt instruction listing the change window times.

What is the most reliable enforcement?

- ☐ A A hook that inspects calls to those tools and blocks them outside the window
- ☐ B Fork the vendor's MCP server and add the check inside each affected tool
- ☐ C Strengthen the system prompt and include the current time in every request
- ☐ D Remove the four production tools from allowedTools permanently

◆ PAUSE AND DECIDE

What is the most reliable enforcement?

- A

 A hook on those tools
Enforcement outside a tool you do not own, evaluating a condition the tool knows nothing about.
- B

 Fork the server
Works, and now you maintain a fork of a vendor product forever. The stem says it cannot be modified.
- C

 Prompt plus current time
Gives the agent the information to comply. It does not make compliance mandatory.
- D

 Remove them from allowedTools
Enforcement, and it is permanent. The rule is time-bound, so this blocks the change window too.

THE SIGNAL

Two enforcement options here, and the difference is conditionality. Permissions are on or off. Hooks can evaluate.

An inventory tool calls a warehouse API. When that API is rate limited it returns HTTP 429; when a SKU does not exist it returns HTTP 404. The tool reports both as "isError: true – warehouse lookup failed".

The orchestration layer retries any failed tool call three times with exponential backoff.

Rate-limit failures usually recover. Unknown-SKU failures consume three retries and a lot of time before the agent moves on.

What is the most effective change?

- ☐ A Reduce the retry count from three to one
- ☐ B Distinguish the two in the error, with a retryable flag and a message naming the cause
- ☐ C Report unknown SKUs as a protocol error so the retry logic skips them
- ☐ D Increase the backoff interval so retries are less costly

◆ PAUSE AND DECIDE

What is the most effective change?

A Fewer retries

Helps the unknown SKU and hurts the rate limit, which needed the retries. It trades one failure for another.

C Protocol error for 404

The tool exists and ran. Misusing protocol errors also hides the message from the model.

B Retryable flag plus cause

The two failures are different, so the results should be different. Each then gets the handling it needs.

D Longer backoff

Makes the wasted retries slower. The unknown SKU is still retried three times.

THE SIGNAL

One error shape for two different failures forces one policy on both. Distinguish at the source, not in the retry rule.

A data pipeline agent calls a transformation tool that occasionally fails on malformed rows. The tool returns `isError` with a clear message naming the offending row and column.

The client code catches the error, logs it, and returns a generic "transformation unavailable" string to the agent as the tool result.

The agent consistently abandons the file rather than skipping the bad row.

What is the root cause?

- ☐ A The agent needs instructions on how to handle transformation failures
- ☐ B The client code replaces an actionable error with a generic one before the model sees it
- ☐ C The transformation tool should skip malformed rows automatically
- ☐ D The error should be a protocol error so the agent treats it as fatal

◆ PAUSE AND DECIDE

What is the root cause?

- A

 Instruct the agent
Instructions cannot compensate for information the agent does not receive. It does not know which row failed.
- B

 Client code swallows the detail
A good error was written and then discarded one layer before the reader who needed it.
- C

 Skip rows automatically
Silently dropping data in a pipeline. It hides a correctness problem rather than surfacing it.
- D

 Make it a protocol error
Backwards. That guarantees the agent sees even less than it does now.

THE SIGNAL

The tool did everything right. The failure is in the layer between the tool and the model.

A customer-facing agent calls an internal credit tool. When the caller lacks permission, the tool returns: "Access denied: account 88213 exists but requires role credit_admin. Query failed at creditdb.internal:5432 in check_limit()."

A security review has flagged the error text. The agent frequently repeats parts of it to the customer when explaining why it cannot help.

What should change?

- ☐ A Instruct the agent never to repeat tool error text to customers
- ☐ B Sanitise the error to state that the operation is not permitted and what to do next, without internal detail or existence disclosure
- ☐ C Return a protocol error so the text never reaches the agent
- ☐ D Remove the error message entirely and return isError with empty content

◆ PAUSE AND DECIDE

What should change?

- A

Instruct the agent not to repeat it

The sensitive text is still in context, and one prompt stands between it and the customer.
- B

Sanitise at the source

Removes the host, the port, the function and the existence disclosure, and still tells the agent what to do next.
- C

Protocol error

Solves disclosure by removing all information. The agent can no longer explain anything or route the customer.
- D

Empty content

Same trade. Safe and useless, and it takes you back to scenario one's stuck agent.

THE SIGNAL

Two things leak here: the infrastructure detail, and the fact that account 88213 exists. The second is the one people miss.